

LogRhythm JSON Parsing Policies: Creating Custom Policies

Overview

LogRhythm's JSON parsing policies outline the procedures for extracting, transforming, and mapping JSON-formatted data sources into a consistent schema.

Policy File Structure

A policy file consists of the following components:

```
1 {
2   "name": "source_name",
3   "filter": "condition_expression",
4   "schemaRule": {
5     "fanout": {
6       "inputField": ["$.path.to.array[*]"]
7     },
8     "convertToJson": ["$.path.to.convert"]
9   },
10  "transforms": [
11    { /* field mappings */ }
12  ],
13  "subtransforms": [
14    { /* conditional transformations */ }
15  ]
16 }
17
```

Core Elements

1. **Name:** This serves as the unique identifier for the data source or policy.
 2. **Filter:** A JPath condition that determines how this policy is applied.
 3. **Schema Rule** (*optional*):
 - **Fanout:** Manages the processing of elements within an array.
 - **Convert to JSON:** Transforms string fields into JSON objects.
 4. **Transforms:** Defines the field mappings from the source schema to the target schema.
 5. **Subtransforms** (*optional*): These are conditional transformation rules that apply under specific circumstances.
-

Field Transformation Structure

Each transform entry follows this format:

```
1  {
2    "inputRule": "$.source.field.path",
3    "LRSchemaField": "normalized_field_name",
4    "type": "String",
5    "default": null,
6    "alternativeFields": [".$.backup.field.path"],
7    "format": null,
8    "FanoutParentElement": "$.path.to.array[*]",
9    "condition": null
10 }
11
```

Transform Properties

- **inputRule**: The JSON path leading to the source field (or a literal value)
 - **LRSchemaField**: The name of the target field
 - **type**: The data type (e.g., string, dateTime, number, decimal.)
 - **default**: The value assigned if the field is absent
 - **alternativeFields**: Fields that serve as backup options
 - **format**: The pattern used for formatting (e.g., applicable for dates)
 - **FanoutParentElement**: The path to the parent array
 - **condition**: The rule for conditional transformation
-

Creating a New Policy

Step 1: Understand Your Data Source

Examine JSON data to identify:

- Important fields
- Nested structures and arrays
- Data types

Step 2: Define Basic Policy Structure

```
1  {
2    "name": "your_source_name",
3    "filter": "@.@metadata.beat == 'yourbeat' || @.device_type =~ /(?!i)your_pattern/",
4    "schemaRule": {
5      "fanout": {
6        "inputField": null
7      }
8    },
9    "transforms": [],
```

```
10   "subtransforms": []
11 }
12
```

Step 3: Define Field Transforms

Example:

```
1 {
2   "inputRule": "$.path.to.your.field",
3   "LRSchemaField": "appropriate_schema_field",
4   "type": "String",
5   "default": null,
6   "alternativeFields": [],
7   "format": null
8 }
9
```

Step 4: Handle Array Data

1. Configure **fanout** rule:

```
1 "schemaRule": {
2   "fanout": {
3     "inputField": ["$.path.to.array[*]"]
4   }
5 }
6
```

2. Use **FanoutParentElement** in transforms:

```
1 {
2   "inputRule": "$.field.in.array",
3   "LRSchemaField": "schema_field",
4   "FanoutParentElement": "$.path.to.array[*]",
5   "type": "String"
6 }
7
```

Step 5: Add Conditional Logic

```
1 "subtransforms": [
2   {
3     "condition": "@.field.exists || @.another.condition",
4     "exitonmatch": true,
5     "transforms": [
6       {
7         "inputRule": "$.conditional.field",
8         "LRSchemaField": "schema_field",
9         "type": "String"
10      }
11    ]
12  }
13 ]
14
```

Step 6: Test and Refine

- Use sample data to verify mappings
- Adjust field paths and conditions as needed

Best Practices

- ✓ Use **JsonPath** syntax for field extraction
- ✓ Include **alternative fields** for resilience
- ✓ Use appropriate **data types and formatting**
- ✓ Use **regex** for case-insensitive matching
- ✓ Apply **filtering conditions** to avoid irrelevant data
- ✓ Leverage **subtransforms** for conditional processing
- ✓ Comment **complex sections** for maintainability

Available Schema Fields

Standard schema fields include:

✚ beatname, device_type, fullyqualifiedbeatname, time, object, objectname, objecttype, hash, policy, result, url, useragent, responsecode, subject, version, command, reason, action, status, sessiontype, process, processid, parentprocessid, parentprocessname, parentprocesspath, quantity, amount, size, rate, minutes, seconds, milliseconds, session, kilobytesin, kilobytesout, kilobytes, packetsin, packetsout, severity, vmid, vendorinfo, threatname, threatid, cve, smac, dmac, sinterface, dinterface, sip, dip, snatip, dnatip, sname, dname, serialnumber, login, account, sender, recipient, group, domainimpacted, domainorigin, protnum, protname, sport, dport, snatport, dnatport, augmented, tag1, tag2, tag3, tag4, tag5, tag6, tag7, tag8, tag9, tag10, original_message

JSON Parsing Policy Reference

Formatters

Formatters allow you to transform values from JSON fields before mapping them to the schema. They are specified in the `format` field of a transform.

Formatter	Data Type	Description	Syntax Example
-----------	-----------	-------------	----------------

lowercase	String	Converts a string to lowercase	<code>"format": "lowercase"</code>
uppercase	String	Converts a string to uppercase	<code>"format": "uppercase"</code>
prefix	String	Adds a prefix to a string	<code>"format": "prefix('PREFIX_')"</code>
suffix	String	Adds a suffix to a string	<code>"format": "suffix('_SUFFIX')"</code>
replace	String	Replaces one substring with another	<code>"format": "replace('old','new')"</code>
replaceall	String	Performs multiple replacements, replaces all instances of old string	<code>"format": "replaceall({'old1','new1'},{'old2','new2'})"</code>
kilobyte	Number	Converts bytes to kilobytes	<code>"format": "kilobyte"</code>
DateTime Formats	Date time	To convert the date/time format, please provide a specific format	<code>"format": "yyyy-MM-ddTHH:mm:ss"</code>

		that you would like to use, as illustrated in the next column. For guidance on creating your own date and time format, please refer to Custom date and time format strings - .NET.	<pre>"format": "MM/dd/yyyy HH:mm:ss" "format": "yyyy-MM-ddTHH:mm:ss.fffK"</pre>
--	--	---	---

Operations

Operations allow you to transform or extract values from JSON fields. They are specified in the `inputRule` field in a transform.

Operation	Data Type	Description	Syntax Example
IsIP	String	Validates if a value is an IP address (IPv4/IPv6)	<pre>"inputRule": "IsIP(\$.sourceIpAddress,true)"</pre>
Regex	String	The method extracts a value using a regex pattern and requires three arguments:	<pre>"inputRule": "Regex(\$.text,pattern=? <groupname"</pre>

		1. The JSON field 2. The regular expression 3. The name of the group from the regex	<pre>>.*),group name)"</pre>
SPLIT	String	Splits a string by a delimiter and returns a specific index	<pre>"inputRule ": "SPLIT(\$.u serIdentit y.principa lId,':',1) "</pre>
Concat	String	Concatenates two or more string values	<pre>"inputRule ": "Concat(\$. firstName, ',\$.lastNa me)"</pre>
ConcatArray	Misc	Joins array elements with a delimiter	<pre>"inputRule ": "ConcatArr ay(\$.recip ients[*],', ',')"</pre>
ToString	Misc	Converts a value to a String.	<pre>"inputRule ":</pre>

			<code>"ToString(\$.numericF ield)"</code>
EpochSectoDate Time	Date Time	Converts Unix timestamp (seconds) to DateTime	<code>"inputRule ": "EpochSect oDateTime(\$.timestam p)"</code>
EpochMilliSect oDateTime	Date Time	Converts Unix timestamp (milliseconds) to DateTime	<code>"inputRule ": "EpochMill iSectoDate Time(\$.tim estamp)"</code>
EpochMicroSec toDateTime	Date Time	Converts Unix timestamp (microseconds) to DateTime	<code>"inputRule ": "EpochMicr oSectoDate Time(\$.tim estamp)"</code>
LocalDateTime	Date Time	Gets the local date-time	<code>"inputRule ": "LocalDate Time()"</code>
Add	Number/Decima l	Adds a number to Json Value	<code>"inputRule ":Add(\$.nu m,1000)</code>

Subtract	Number/Decimal	Subtracts a number from JSON value	"inputRule": "subtract(\$.num,10)"
Multiply	Number/Decimal	Multiply Json Value by a number	"inputRule": "multiply(\$.num,25)"
Divide	Number/Decimal	Divide Json Value by a number	"inputRule": "divide(\$.bytes,1024)"

Example Usage

Example 1: Basic Field Extraction

```

1 {
2   "inputRule": "$.sourceIPAddress",
3   "LRSchemaField": "sip",
4   "type": "string",
5   "default": null,
6   "alternativeFields": ["$.src_ip", "$.source.ip"],
7   "format": null
8 }
9

```

Example 2: Using an Operation

```

1 {
2   "inputRule": "IsIP($.sourceIPAddress,true)",
3   "LRSchemaField": "sip",
4   "type": "string",
5   "default": null,
6   "alternativeFields": null,
7   "format": null
8 }
9

```

Example 3: Using a Format

```

1 {
2   "inputRule": "$.username",
3   "LRSchemaField": "login",

```

```
4  "type": "string",
5  "default": null,
6  "alternativeFields": null,
7  "format": "lowercase"
8  }
9
```

Example 4: Using Both Format and Operation

```
1  {
2  "inputRule": "SPLIT($.userIdentity.principalId,':',1)",
3  "LRSchemaField": "session",
4  "type": "string",
5  "default": null,
6  "alternativeFields": ["$.userIdentity.arn"],
7  "format": "lowercase"
8  }
9
```

Example 5: DateTime Formatting

```
1  {
2  "inputRule": "$.eventTime",
3  "LRSchemaField": "normal_msg_date",
4  "type": "datetime",
5  "default": null,
6  "alternativeFields": null,
7  "format": "yyyy-MM-ddTHH:mm:ss.fffK"
8  }
9
```

Example 6: Using Array Concatenation

```
1  {
2  "inputRule": "ConcatArray($.response.messagesDelivered.recipient[*],',')",
3  "LRSchemaField": "recipient",
4  "type": "String",
5  "default": null,
6  "alternativeFields": null,
7  "format": null
8  }
9
```
